has a 4-hour MTTR, that team then has a maintainable release rating of 40. To compare this to a bad number, if you have a team that has a hot fix with every release and an MTTR of 12 hours, that would give you a maintainability release rating of 720. Due to the broad range of potential values, this metric becomes a good benchmark that your team can focus on improving over time rather than targeting a specific value out of the gate. Table 8.1 shows example inputs and outputs for the maintainable release rating (MRR).

Table 8.1. Example inputs and outputs for a maintainable release rating

Inputs	MRR	Notes
MTTR = 240 minutes Total fixes = 3 Total releases = 30	24	Given a 4-hour release time, this team needs to fix only 1 in every 3 releases, so the rating isn't bad.
MTTR = 240 minutes Total fixes = 10 Total releases = 5	480	Even though this team has a 4-hour release time, they have 2 fixes per release on average, which is really bad.
MTTR = 480 minutes Total fixes = 1 Total releases = 100	4.8	This team has a higher MTTR, but they rarely have to fix things (once in 100 releases), so they're doing great.

8.3.3. Code coverage

Code coverage is the percentage of your codebase that's covered by automated tests. You can measure code coverage during the build process with a number of tools that run during the build. Some examples are Cobertura, JaCoCo, Clover, NCover, and Gcov. I'll take advantage of the comprehensive dashboards in SonarQube to show you how to turn code coverage reports into actionable data. An example coverage report is shown in figure 8.9.

Figure 8.9. An example breakdown of code coverage in SonarQube